



SOUTENANCE PROJET 11

Fruits! — Chaîne de traitement Big Data

Parcours AI Engineer - Preprocessing, transfer learning et réduction de dimension avec PySpark — vers une architecture Big Data sur AWS

Stéphanie Duhem - Juillet 2026



00. CONTEXTE & OBJECTIFS

LE CONTEXTE

- **Start-up AgriTech** : développement de robots cueilleurs intelligents pour la préservation de la biodiversité
- **Visibilité** : application mobile de reconnaissance de fruits › sensibilisation à la biodiversité et amorce d'architecture Big Data
- **Mon rôle** : Pipeline de featurisation en amont du futur modèle de classification et adaptée Big Data + Cloud

OBJECTIFS

- › Reprise de l'existant : Chaîne de traitement pour l'app Fruits! réalisé en optique Cloud 
- › Consolidation : Finaliser et optimiser la chaîne de traitement dans une optique Big Data 
- › Preuve de concept : Démontrer la faisabilité d'un passage à l'échelle sur une architecture Big Data + Cloud 



CONTRAINTES TECHNIQUES

- **Volume** : Jeu de données en croissance (90 483 images aujourd'hui) 
- **RGPD** : Traitement et stockage sur territoire européen 
- **Calcul distribué** : PySpark 
- **Budget** : Ressources Cloud maîtrisées 

01. DATASET – Fruits-360 (Kaggle)

LE DATASET COMPLET

- **Volume** : 90 483 images (train + test) - 131 classes
- **Format des images** : 100×100 pixels - RGB - JPEG

```
fruits-360/  
├── Training/      (131 classes)  
│   └── Apricot/0_100.jpg, 1_100.jpg, ...  
└── Test/          (131 classes, même structure)
```

Dimensions du dataset à adapter à l'usage

(Mise en place de la pipeline sans training)

» **Méthode** : Échantillonnage systématique > diversité représentative



```
data/sample/  
├── Apricot/30_100.jpg, 31_100.jpg, ...  
├── Cherry/...  
└── ... (10 classes au total, pas de split  
Training/Test)
```

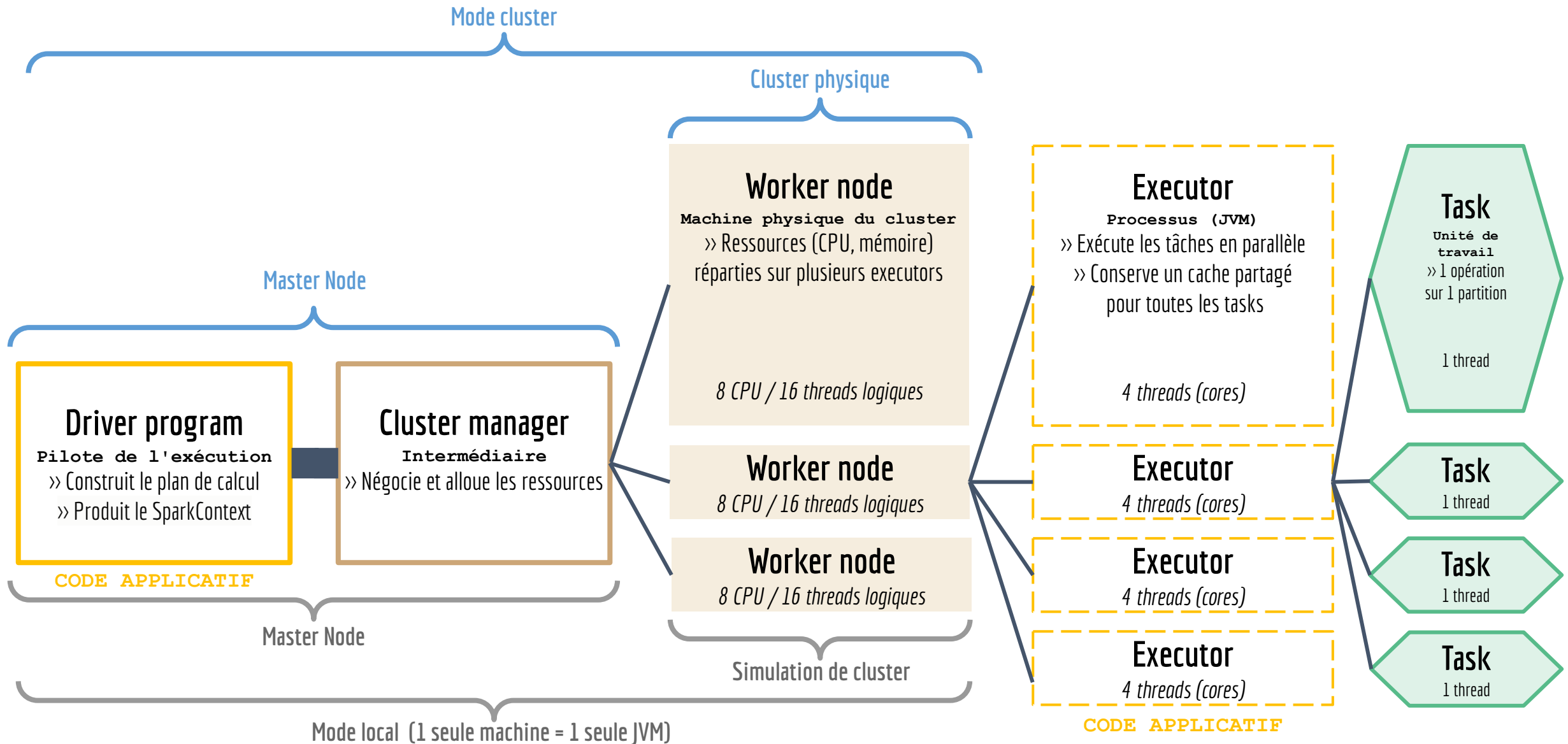
ÉCHANTILLON DE TRAVAIL

Source : train (prepare_sample.py)

300 images - 10 classes

- Apple Braeburn 
- Apricot 
- Cherry 
- Dates 
- Guava 
- Mango Red 
- Papaya 
- Pear Williams 
- Plum 
- Strawberry 

02-A. SPARK - Moteur de calcul distribué en 4 acteurs



02-B. PySPARK - Chaîne de traitement pour calcul distribué

1. Chargement

Fruits-360 (le dataset)

- » Récupération des métadonnées et du contenu (Chemin, taille .. + contenu brut en bytes)
- » Label extrait du path
- » Aucun décodage

2. Modèle

MobileNetV2

- » Transfert Learning (ImageNet)
- » Extraction des features sous forme d'un vecteur de 1280 valeurs par image (avant-dernière couche)
- » Sans couche de sortie (classification)
- » Récupération des poids du modèle par broadcast (pas par sérialisation du modèle entier)
- » Optimisation du code existant

3. Featurisation

pandas UDF

- » Traitement par lots successifs (pas ligne par ligne)
- » Lots transférés via Arrow
- » Lots traités par le code applicatif
- » Modèle chargé une fois par task, réutilisé pour tous les lots de cette task

4. Réduction de dimension

PCA distribuée

- » Réduction des dimensions des vecteurs (1280 → 53)
- » 90,1% de variance conservée.
- » Covariance calculée en distribué
- » Décomposition finale sur le driver (peu coûteuse)

5. Sauvegarde

Format Parquet

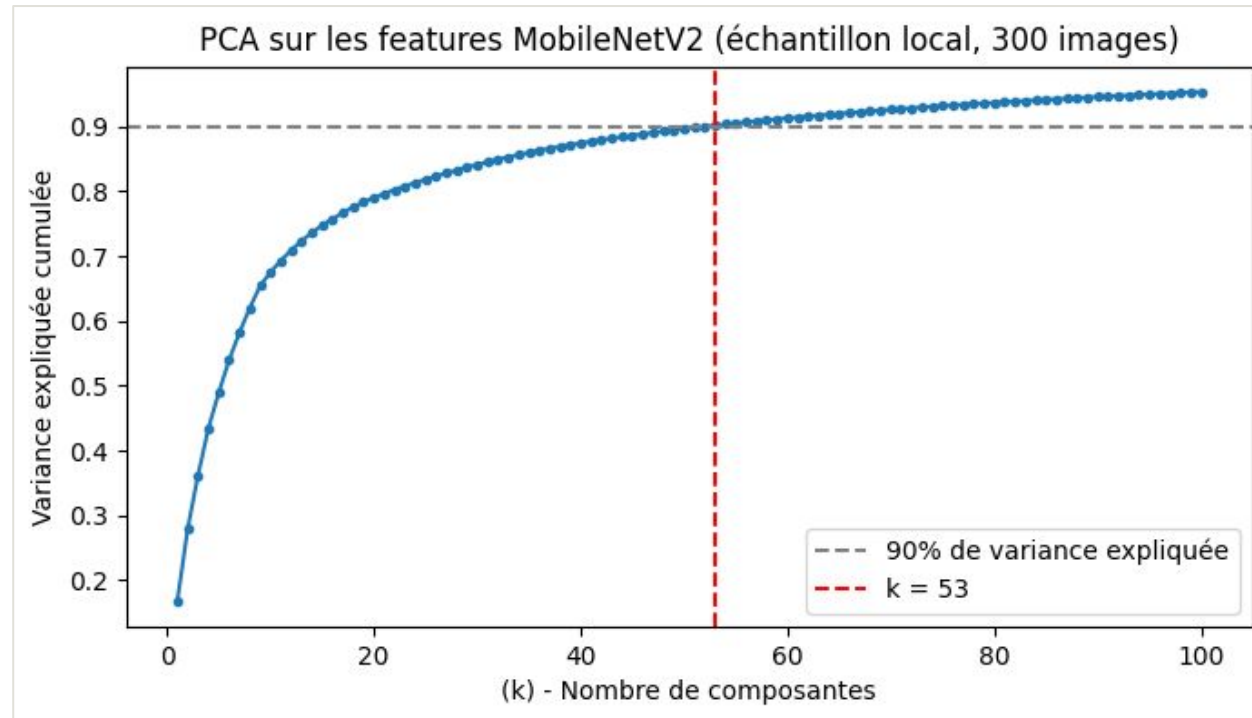
- » Sélection de path, label & pca_features (vecteur converti en array)
- » Lisible hors Spark
- » Résultat 95,9% de réduction de taille par rapport au vecteur MobileNetV2 initial.

Pourquoi le broadcast des poids du modèle ?

`feature_model` est un objet complexe (graphe TensorFlow), impossible à sérialiser efficacement vers chaque processus à chaque tâche.

» **Solution** : chaque processus reconstruit l'architecture localement et reçoit uniquement les poids déjà entraînés via broadcast

02-C. PySpark - Exécution en local



	path	label
0	file:/Users/stephanieduhem/Documents/_DIPLOMES...	Strawberry Wedge
1	file:/Users/stephanieduhem/Documents/_DIPLOMES...	Strawberry Wedge
2	file:/Users/stephanieduhem/Documents/_DIPLOMES...	Strawberry Wedge
3	file:/Users/stephanieduhem/Documents/_DIPLOMES...	Guava
4	file:/Users/stephanieduhem/Documents/_DIPLOMES...	Guava

	pca_features
0	[-5.287437330475509, 0.9938178276457581, 20.97...
1	[-2.5941090307530086, 0.32458644373422624, 19....
2	[-5.224705457906081, 1.0375195715629957, 19.60...
3	[-4.891880303929059, -7.5702791631416995, 4.84...
4	[-6.323576325095195, -8.627868361164134, 3.570...

Dimension du vecteur PCA : (53,)

Réduction de dimension : 1280 -> 53 (95.9% de moins)

03-A. AWS - Les briques choisies et leur rôle

EMRFS (EMR File System)

Lecture : `spark.read...load("s3://...")` & Écriture : `df.write.parquet("s3://...")`

Bucket S3

Stockage indépendant des données
Région eu-north-1 (Stockholm, Suède) - Maintenu (stateless)

Données

» Images sources et des résultats (matrice PCA)

Vigilance RGPD

» Point de vigilance identifié : le bucket S3 du code existant n'était pas en région UE
» S3 et EMR doivent être dans la même région pour rester cohérents et éviter des coûts/latences de transfert inter-région.

EMR

Cluster Spark managé, Elastic MapReduce
Région eu-north-1 (Stockholm, Suède) - Résilié en fin de tâche

Master Node > Primary Node = 1 Instance EC2

» Héberge le driver program **Spark** et le gestionnaire de ressources (CPU, mémoire) **YARN**

Worker node > Core Node & Task Node

» Héberge les conteneurs YARN, chacun hébergeant un executor
» Continuité avec le code existant

IAM

Gestion des identités et permissions, Identity and Access Management

Utilisateur IAM (p11-admin) » identité permanente, accès au bucket via la console

Rôle IAM (EMR_EC2_DefaultRole) » attachée à une Policy IAM, liste les actions autorisées sur un bucket, persistant

Instance profile » rôle IAM rattaché à l'instance EC2 du cluster, récupère la Policy IAM, résilié en fin de tâche

Sécurité » Aucune permission n'est accordée par défaut entre deux services AWS : sans rôle IAM explicite, chaque tentative de lecture S3 depuis EMR échouerait (Access Denied).

03-B. AWS - Déploiement

DÉPLOIEMENT INITIÉ

- » **Compte AWS** créé, budget avec alertes (50/80/100%)
- » **Utilisateur IAM** dédié (hors compte root)
- » **Bucket S3** en région **UE**, chiffré, accès public bloqué
- » **Données** de test uploadées sur S3
- » **Portabilité vers EMR** : Script de bootstrap EMR + notebook cloud adaptés (chemins S3), prêts à l'usage

eu-north-1.console.aws.amazon.com/s3/home?region=eu-north-1#

Amazon S3

Amazon S3

Compartiments

Compartiments à usage général

Compartiments de répertoires

Compartiments de table

Compartiments de vecteur

Fichiers

Systèmes de fichiers

Gestion des accès et sécurité

Points d'accès

Points d'accès pour FSx

Access Grants

Compartiments

Compartiments à usage général

Toutes les régions AWS

Compartiments de répertoires

Compartiments à usage général (1) Infos

Créer un compartiment

Les compartiments sont des conteneurs pour les données stockées dans S3.

Rechercher des compartiments par nom

Nom	Région AWS	Date de création
p11-fruits-bigdata-sd2026-276005772614-eu-north-1-an	Europe (Stockholm) eu-north-1	25 Jul 2026 08:13:44 PM CEST

p11-fruits-bigdata-sd2026-276005772614-eu-

Objets

Métadonnées

Propriétés

Autorisations

Objets (2)

Les objets sont les entités fondamentales stockées dans Amazon S3. Vous pouvez accéder à vos objets, vous devez leur accorder explicitement des autorisations.

Rechercher des objets en fonction du préfixe

Nom	Type
bootstrap/	Dossier
sample/	Dossier

p11-fruits-bigdata-sd2026-276005772614-eu-north-1-an

sample/

Objets

Propriétés

Objets (10)

Les objets sont les entités fondamentales stockées dans Amazon S3. Vous pouvez accéder à vos objets, vous devez leur accorder explicitement des autorisations.

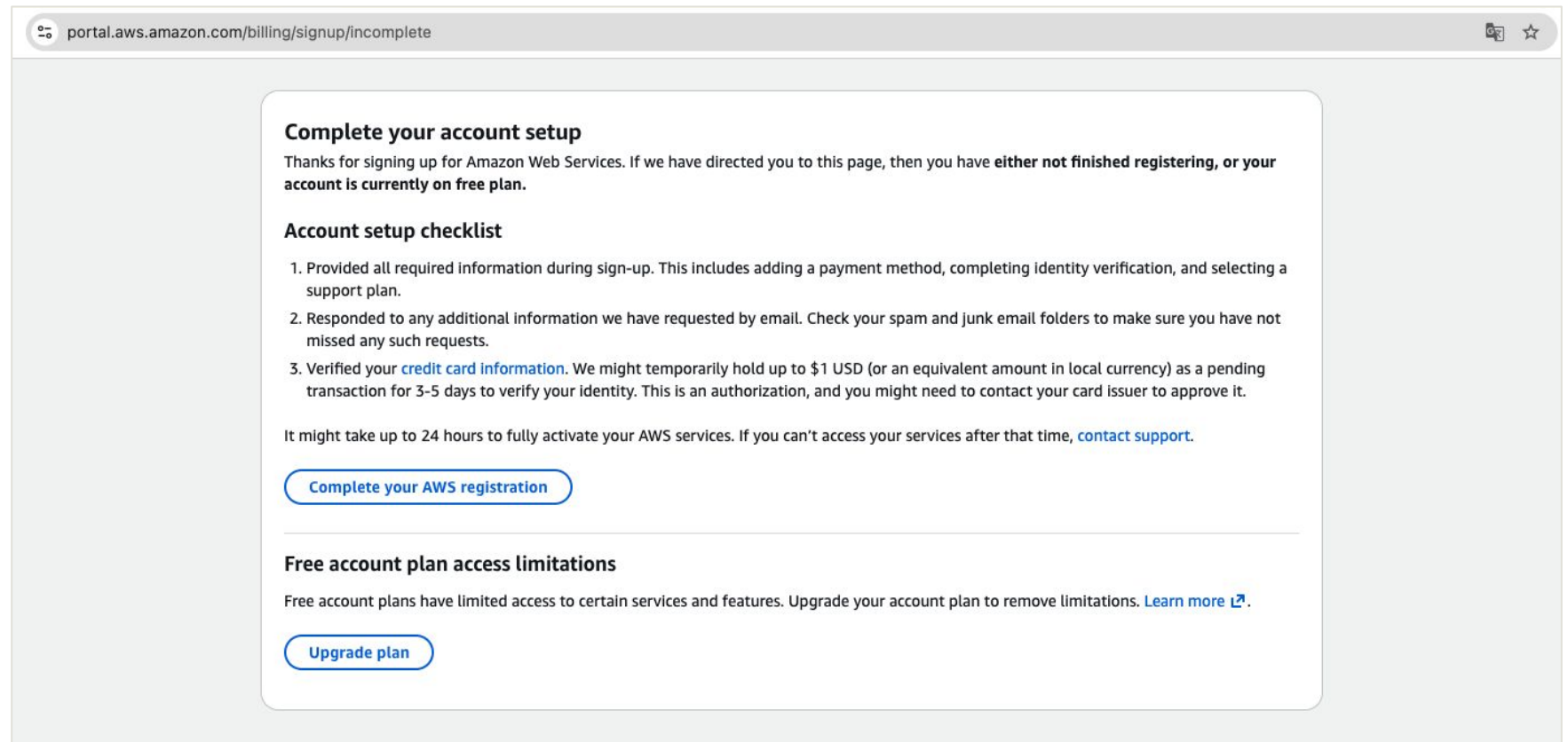
Rechercher des objets en fonction du préfixe

Nom	Type
Apple Braeburn/	Dossier
Apricot/	Dossier
Cherry 1/	Dossier
Dates/	Dossier
Guava/	Dossier
Mango Red/	Dossier

03-C. AWS - Blocage plateforme

BLOCAGE TECHNIQUE AWS

- Le compte est placé sur un "Free account plan" avec accès restreint : EMR reste bloqué tant que la vérification n'est pas complète.
- La vérification n'aboutit pas > **contrainte de la plateforme.**



04. BILAN & PERSPECTIVES

OBJECTIFS ATTEINTS

- » **Présentation de l'architecture Spark** et de sa portabilité vers EMR
- » Ajout des 2 briques demandées : **broadcast des poids & PCA**
- » **Pipeline PySpark** locale complète, testée et validée de bout en bout
- » Démarches de mise en oeuvre de l'environnement **Big Data**
- » **Présentation & mise en place de l'infrastructure AWS** (compte, IAM, S3, RGPD, bootstrap, portabilité EMR avec notebook cloud)
- » **Consolidation** des travaux existants

LIMITES & PROCHAINES ÉTAPES

- Finaliser le déploiement EMR dès la levée du blocage AWS
- Revalider le k de la PCA sur le jeu de données complet (131 classes)
- Recalculer le partitionnement selon les executors réels du cluster



Merci de votre attention.

